

ATIVIDADE AVALIATIVA

Bibliotecas para Interfaces Gráficas em Python

Pesquisa, Apresentação e Demonstração Prática

Formato	Duração por grupo	Nota total
Apresentação em grupo	12 minutos	10 pontos
Modalidade	Prazo de entrega	Entrega do código
Pesquisa + Prática	Verificar no Teams	Junto à apresentação

1. Contexto e Objetivo

O desenvolvimento de interfaces gráficas é uma habilidade essencial para qualquer desenvolvedor Python que deseja criar aplicações acessíveis e interativas. O ecossistema Python oferece diversas bibliotecas para esse fim, cada uma com filosofia, casos de uso e nível de complexidade distintos.

Nesta atividade, cada grupo receberá uma biblioteca para pesquisar, aprender e apresentar para a turma. O objetivo não é apenas mostrar que o grupo conseguiu utilizar a ferramenta, mas ensinar os colegas sobre ela com profundidade técnica e exemplos reais e funcionais.

2. Distribuição por Grupo

Cada grupo ficará responsável por uma das bibliotecas listadas abaixo. A distribuição será definida pelo professor:

Grupo	Biblioteca	Descrição Resumida	Grupo
01	Tkinter e CustomTkinter	Biblioteca nativa do Python (sem instalação). CustomTkinter adiciona widgets modernos sobre ela com suporte a temas dark/light.	Samela
02	PyQt6	Wrapper Python do framework Qt. Muito utilizado em aplicações desktop profissionais, com suporte a GUI Designer e sistema de Signals & Slots.	Thiago
03	Kivy	Biblioteca open-source focada em aplicações multi-touch e multiplataforma. Suporta execução em desktop, Android e iOS.	João Gabriel
04	Flet	Permite criar aplicações Flutter (web, desktop e mobile) escrevendo apenas Python. Baseado em componentes Material Design.	Pedro
05	Streamlit	Focada em ciência de dados e IA. Cria aplicações web interativas com poucas linhas de código, ideal para dashboards e visualizações.	Klaivyn

3. Requisitos da Apresentação

Todos os grupos devem obrigatoriamente abordar os seguintes tópicos durante a apresentação, independentemente da biblioteca escolhida. O tempo de apresentação é de 12 minutos por grupo — planejem o tempo com antecedência.

3.1 Apresentação da Biblioteca

- O que é a biblioteca, quem a mantém e desde quando existe.
- Por que foi criada e quais problemas ela resolve.
- Em quais cenários e mercados é mais utilizada.

3.2 Instalação e Importação

- Como instalar a biblioteca via pip (ou outro gerenciador de pacotes).
- Quais importações são necessárias para iniciar o desenvolvimento.
- Apresentar o código mínimo para criar a primeira janela ou tela.

3.3 Estrutura Básica de uma Tela

- Explicar como a biblioteca organiza seus componentes: janelas, widgets, layouts e eventos.
- Mostrar um esqueleto de código com explicações de cada parte.
- Destacar os principais widgets utilizados nas demonstrações.

3.4 Vantagens e Desvantagens

- Indicar claramente quando utilizar — e quando NÃO utilizar — a biblioteca.

3.5 Demonstrações Práticas

Todos os grupos irão implementar o mesmo projeto: um **aplicativo de Lista de Tarefas**. As duas telas a seguir devem ser desenvolvidas com a biblioteca do grupo e demonstradas funcionando ao vivo durante a apresentação.

Projeto: Aplicativo de Lista de Tarefas

O armazenamento dos dados deve ser feito em memória, utilizando uma lista de dicionários Python (sem banco de dados ou arquivos externos). Ao encerrar o programa, os dados não precisam ser persistidos.

Tela 1 — Exibir Tarefas Cadastradas

- Listar todas as tarefas cadastradas (título e status).
- Permitir marcar uma tarefa como concluída diretamente na lista.
- Exibir indicação visual diferenciada para tarefas concluídas e pendentes.
- Ter botão ou ação para navegar até a Tela 2 (Adicionar Tarefa).

Tela 2 — Adicionar Tarefa

- Campo de texto para o título da tarefa.
- Campo de texto para descrição/detalhes (opcional).
- Botão Salvar que insere a tarefa na lista em memória e retorna para a Tela 1.
- Botão Cancelar que descarta a entrada e retorna para a Tela 1.
- Validação básica: não permitir salvar tarefa com título vazio.

- O código de ambas as telas deve ser mostrado e explicado durante a apresentação — não apenas executado.
- O comparativo entre as implementações dos grupos é parte do aprendizado da turma.

3.6 Fontes e Recursos

- Indicar onde encontrar a documentação oficial da biblioteca.
- Recomendar tutoriais, fóruns ou comunidades ativas para quem quiser aprofundar o conhecimento.
- O grupo deve ser capaz de indicar o melhor caminho para um colega aprender a biblioteca após a aula.

4. Critérios de Avaliação

A nota é parcialmente individual dentro do grupo — a participação e o domínio do conteúdo de cada integrante serão observados. Apresentações em que apenas um membro fala terão desconto na nota individual dos demais.

Critério	O que será avaliado	Peso
Domínio técnico	O grupo demonstra entender a biblioteca além da superfície? Consegue responder perguntas da turma e do professor?	2,5 pts
Qualidade do código	Código limpo, organizado e comentado. Boas práticas da linguagem e da biblioteca.	2,0 pts
Demonstração das telas	As duas telas funcionam ao vivo. Cada componente é explicado com clareza e didática.	2,5 pts
Análise crítica	Vantagens e desvantagens apresentadas com embasamento real, não apenas opiniões genéricas.	1,5 pts
Didática e clareza	Apresentação compreensível para quem nunca usou a biblioteca. Participação equilibrada entre os membros.	1,0 pt
Tempo e organização	Respeita o limite de 12 minutos. Apresentação com início, meio e fim bem definidos.	0,5 pt
TOTAL		10 pts

5. Prazo e Forma de Entrega

5.1 Prazo

Verificar no Teams.

5.2 O que deve ser entregue

1. Arquivo(s) .py com o código das duas telas (Tela 1 e Tela 2) funcionando corretamente.
2. Os arquivos devem estar nomeados de forma clara (ex: tela_lista.py, tela_adicionar.py).

3. A entrega deve ser realizada pela plataforma da disciplina até o horário de início da apresentação do grupo.

5.3 Formato da Apresentação

- Duração máxima: 12 minutos por grupo. Ao término, a turma terá 3 minutos para perguntas.
- O código deve ser apresentado em editor de código (VSCode, PyCharm ou similar) e executado ao vivo.
- Slides são opcionais — a demonstração prática tem mais peso que a apresentação teórica.

6. Orientações Importantes

- Teste o código em pelo menos dois computadores diferentes antes da apresentação. Problemas de ambiente eliminam pontos desnecessariamente.
- Seja específico nas vantagens e desvantagens. "É fácil de usar" não é uma vantagem — "permite criar uma janela funcional em menos de 10 linhas" é.
- Divida claramente a apresentação entre os membros. Cada integrante deve ter uma parte definida para apresentar.